
Waiting for everyone to join
TU257
Information Systems
Marisa Llorens

Week 12

IS

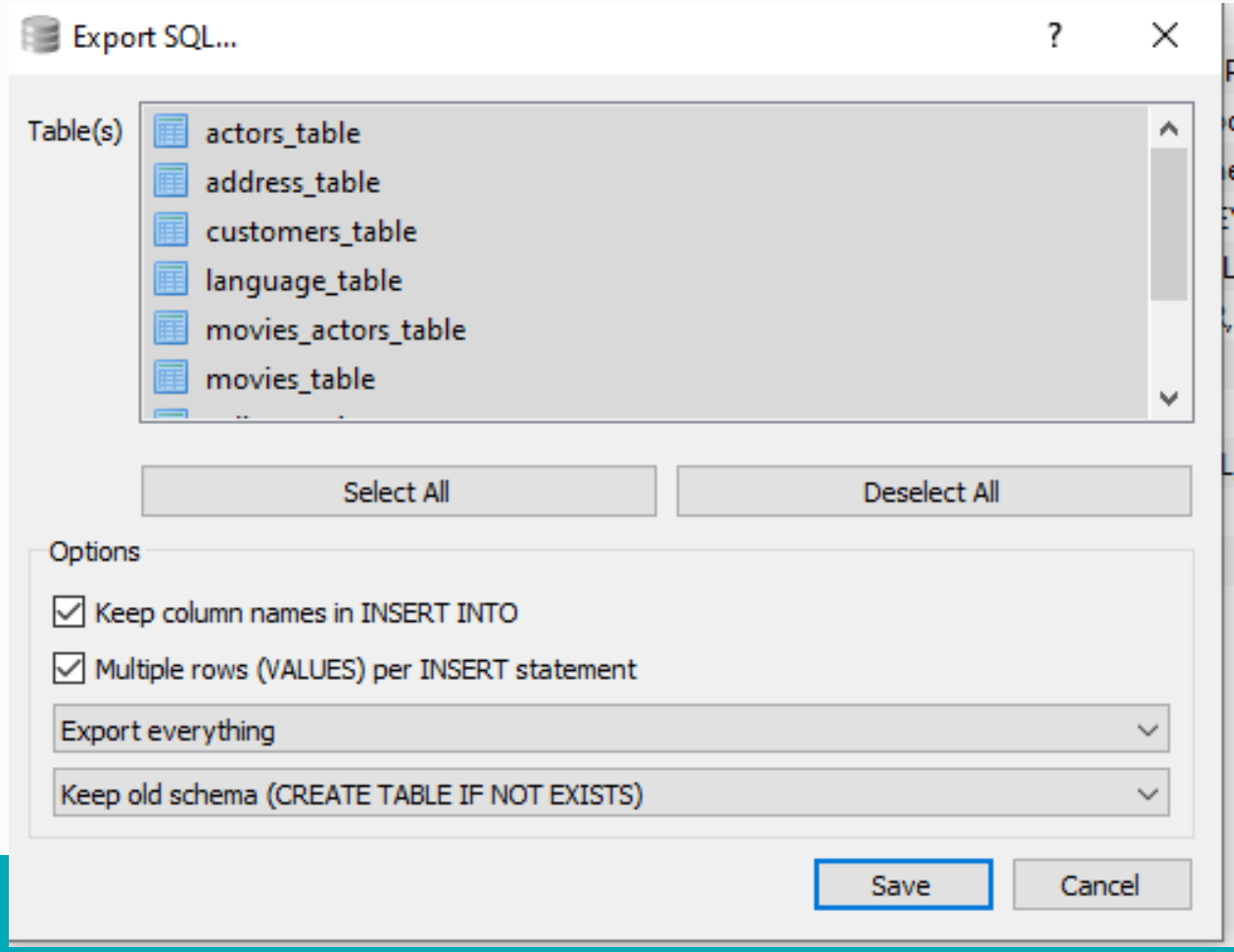
TU257

Agenda

- Back up/Dump Files
- Optimisation
- More Subqueries

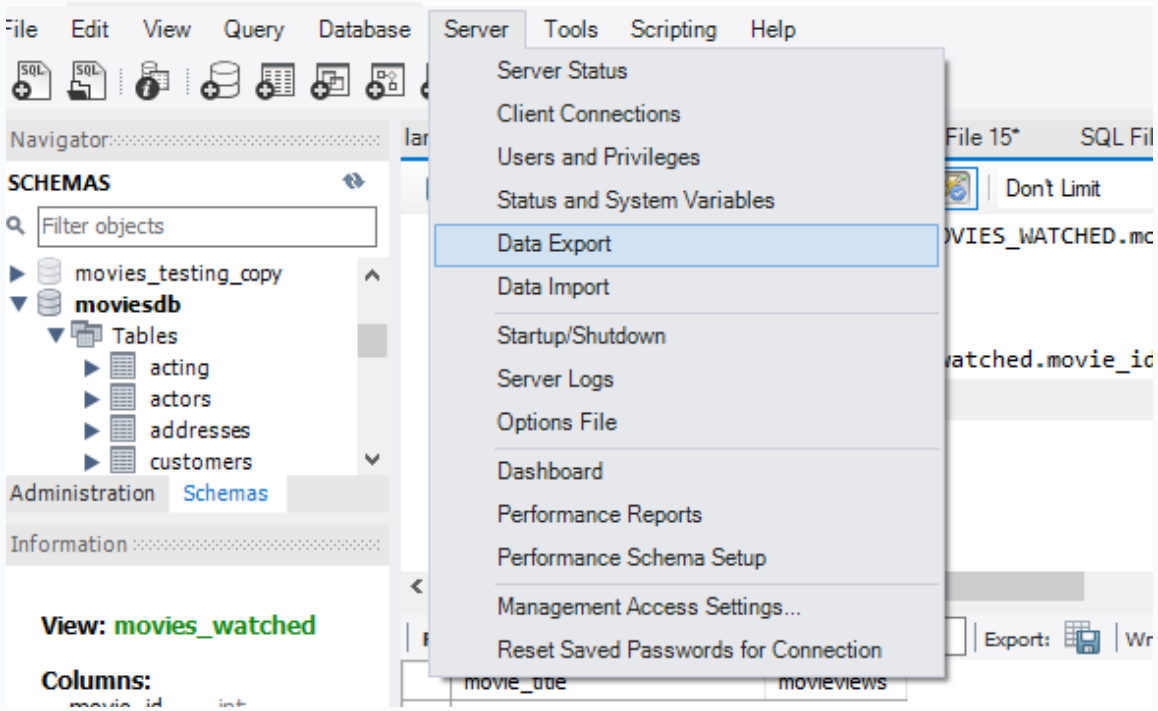
Back up files
Dumps

- We can create back ups for:
  - Schema
  - Data
  - Schema and data
- Exporting tool



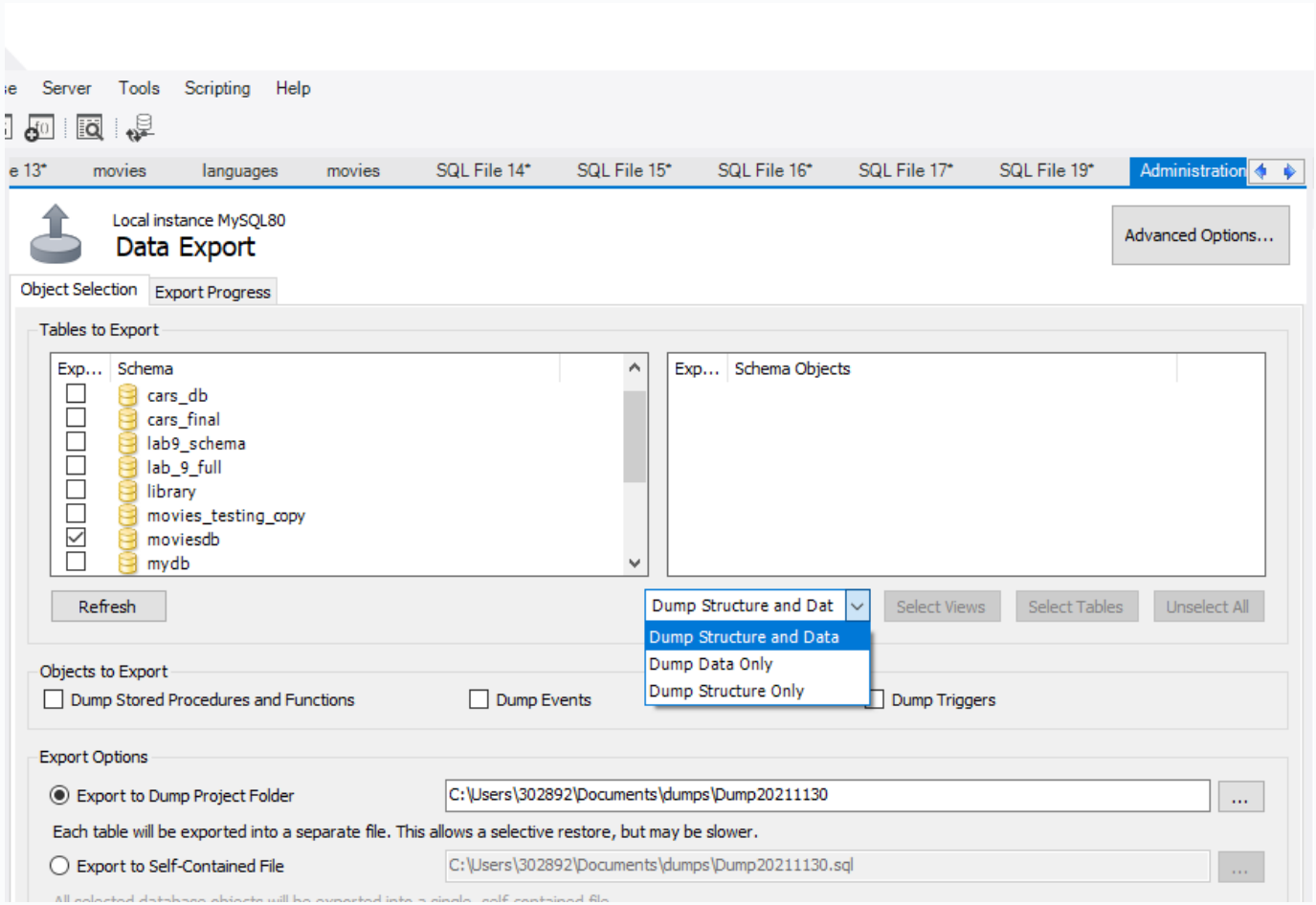
MySQL Backups Dumps

- Creating dump files
- Server
    - Data Export



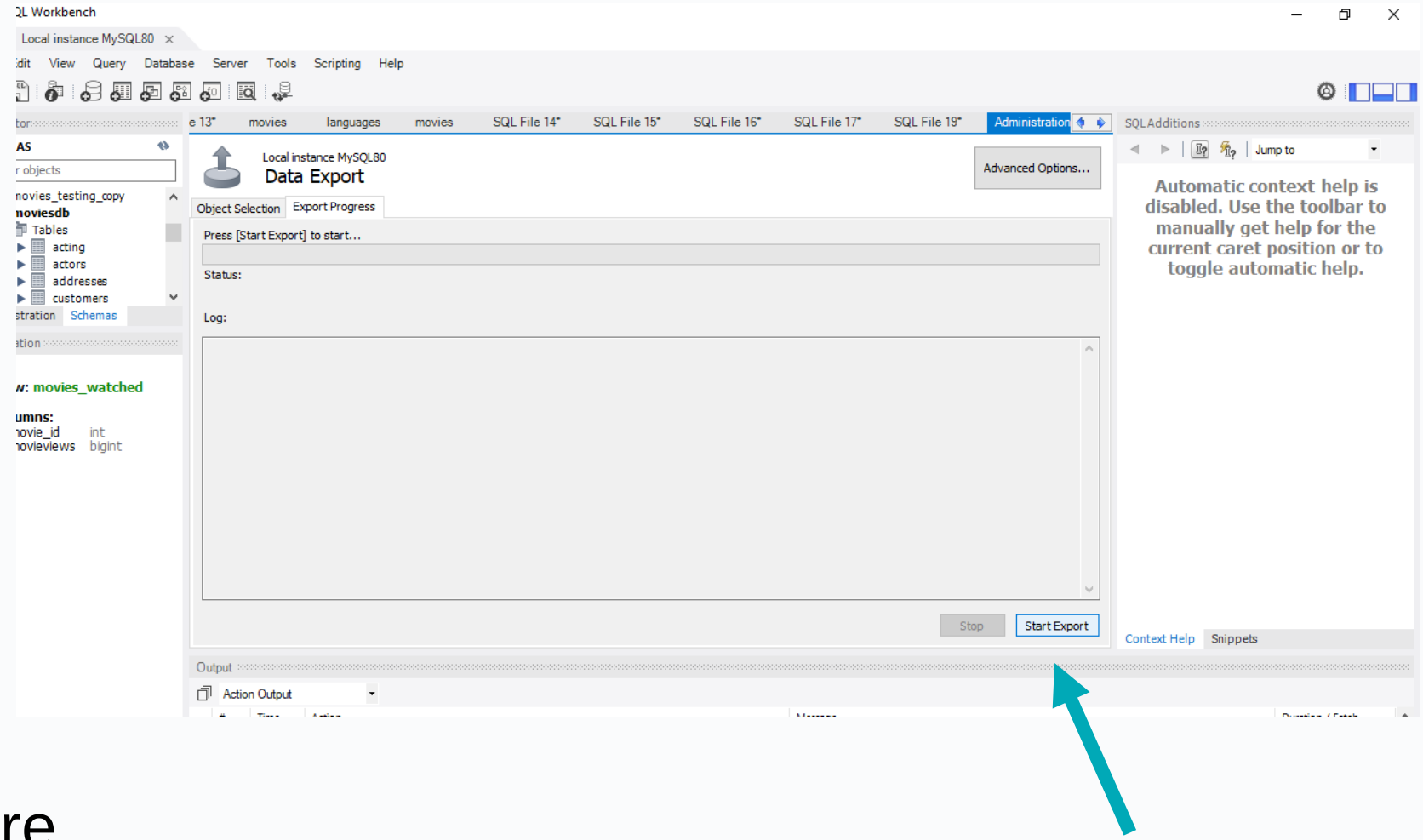
Backups
Dumps

- Creating dump files
- Data Export
  - Data only
  - Structure only
  - Data and Structure
- Project folder or
- Self contained file



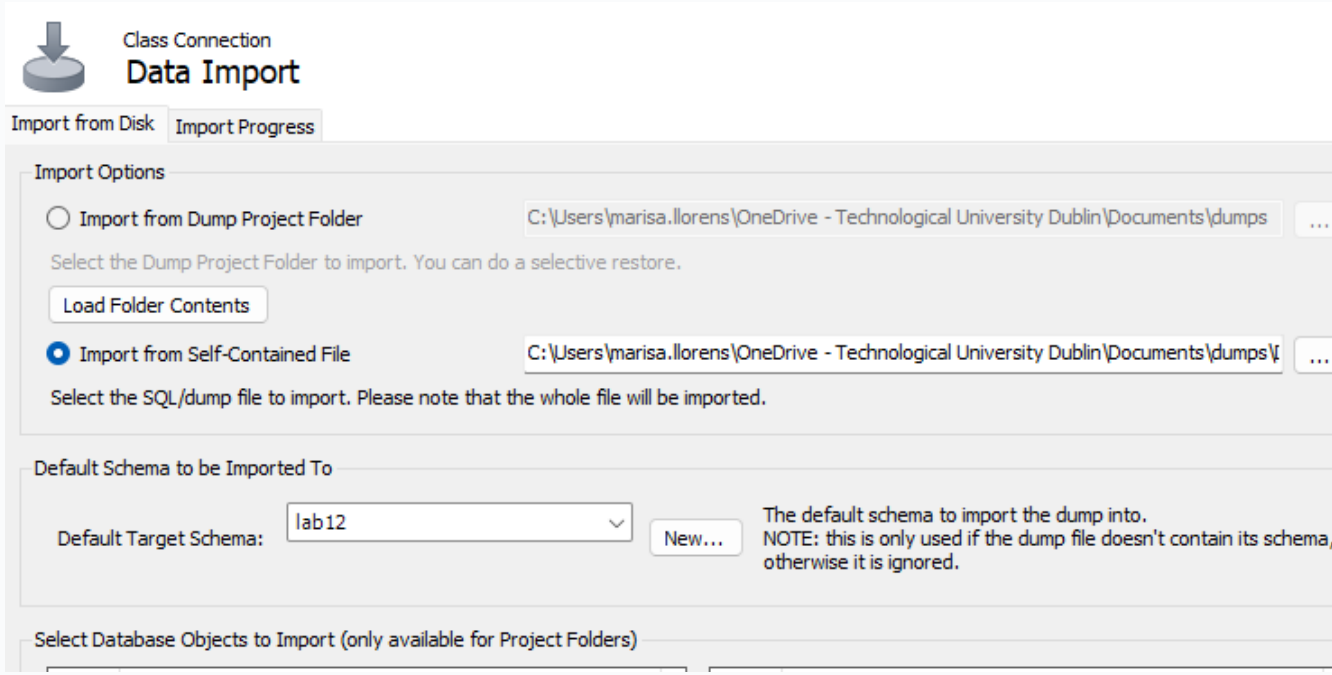
Backups Dumps

- Creating dump files
- Data Export
  - Data only
  - Structure only
  - Data and Structure



Restoring DBs Dumps

- Download the dump file from Brightspace
- Open MySQL workbench
- Create a new schema called lab12
- Select Server-> Data Import



- Select Self-Contained file
- Select path to dump
- Select target schema as lab12

Optimisation and Indexes

Indexes

- Indexes are data structures that help retrieve row data with specific column values faster.

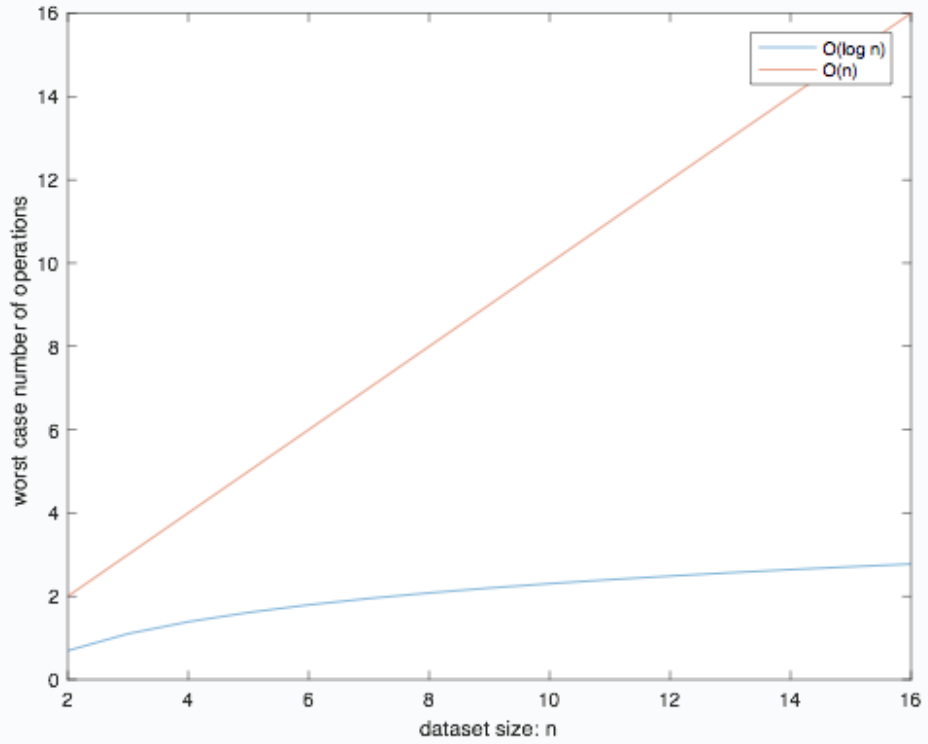
- Without an index, the search must begin with the first row and then read through the entire table to find the relevant rows. The larger the table, the more this costs.

Indexes

- Indexes can especially improve performance for larger databases.

- Note that each index decreases update/insert/delete performance, so you should only have them where they're actually needed.

Indexes



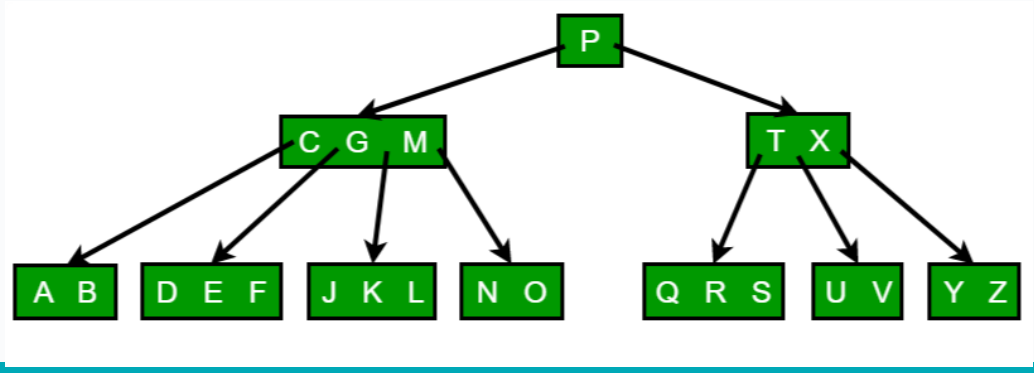
- O(n) No order– No index
- O(log(n)) Ordered data-- Index

https://medium.com/@JasonWyatt/squeezing-performance-from-sqlite-indexes-indexes-c4e175f3c346

Indexes

- If the table has an index for the columns in question, the database can quickly determine the position to seek to in the middle of the data file without having to look at all the data. This is much faster than reading every row sequentially.

- Most indexes are stored in B-Trees



Optimisation Summary

- Indexes increase efficiency and optimize retrieval time but slow down other tasks such as
  - Update
  - Insert

Optimisation Summary

- Indexes should not be used in:
  - Small tables.
  - Tables that have frequent, large batch update or insert operations.
  - Columns that contain a high number of NULL values.
  - Columns that are frequently manipulated.

MySQL Index specifics

MySQL: Types of Indexes

- When you create a table with a primary key, MySQL automatically creates a special index named PRIMARY.
- This index is called the clustered index.
- The PRIMARY index is special because the index itself is stored together with the data in the same table.
- Other indexes other than the primary index are called secondary indexes or non-clustered indexes.

MySQL: Types of Indexes

- A **primary key** is an index for which each index value differs from every other and uniquely identifies a single row in the table.
  - A primary key cannot contain `Null` values.
  - In the InnoDB engine the data is physically organized for ultra-fast lookups and sorts based on PK.
- A **unique** index is similar to a primary key, except that it can be allowed to contain `Null` values.

MySQL: Types of Indexes

- A **non-unique** index is an index in which any key value may occur multiple times.

MySQL: Types of Indexes

- Single column
  - INDEX(col1)
- Multicolumn
  - INDEX(col1, col2, col3) will also index [col1] and [col1,col2]
  - Searches frequently performed on the same set of multiple columns (e.g., first and last name, city and state, etc.)

MySQL: Optimisation

- Index columns that you want to search by. When you create the table:

```
CREATE TABLE t(
  c1 INT PRIMARY KEY,
  c2 INT NOT NULL,
  c3 VARCHAR(10),
  INDEX (c2,c3) );
```

MySQL: Optimisation

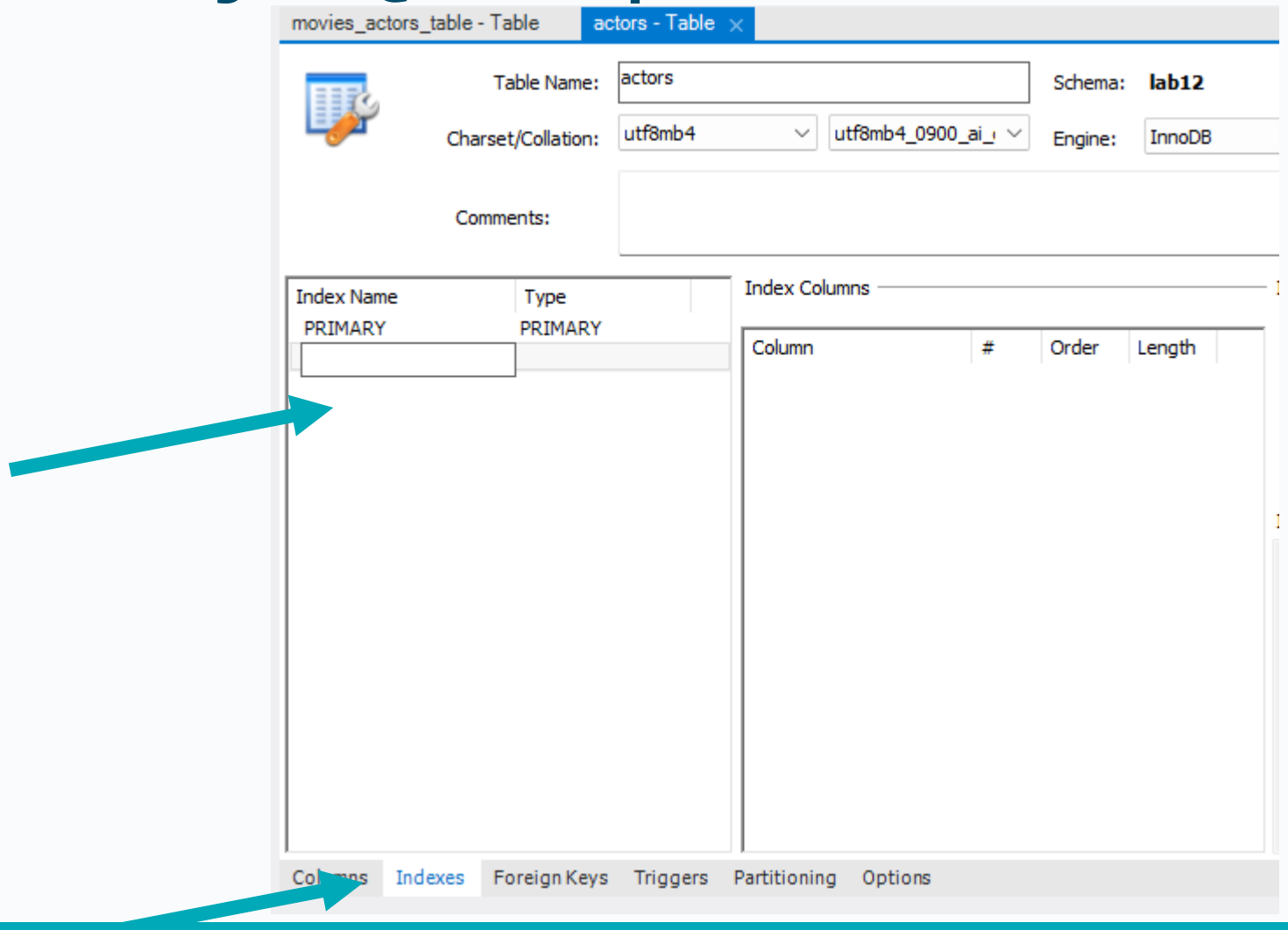
After the table was created

```
ALTER table t ADD INDEX (c4);
```

or

```
CREATE INDEX idx_c4 ON t(c4);
```

MySQL: Optimisation



MySQL: Optimisation

- **`EXPLAIN`**
  - Displays information about the order and structure of a **`SELECT`** statement.
  - Can be used to see where keys are not being used efficiently

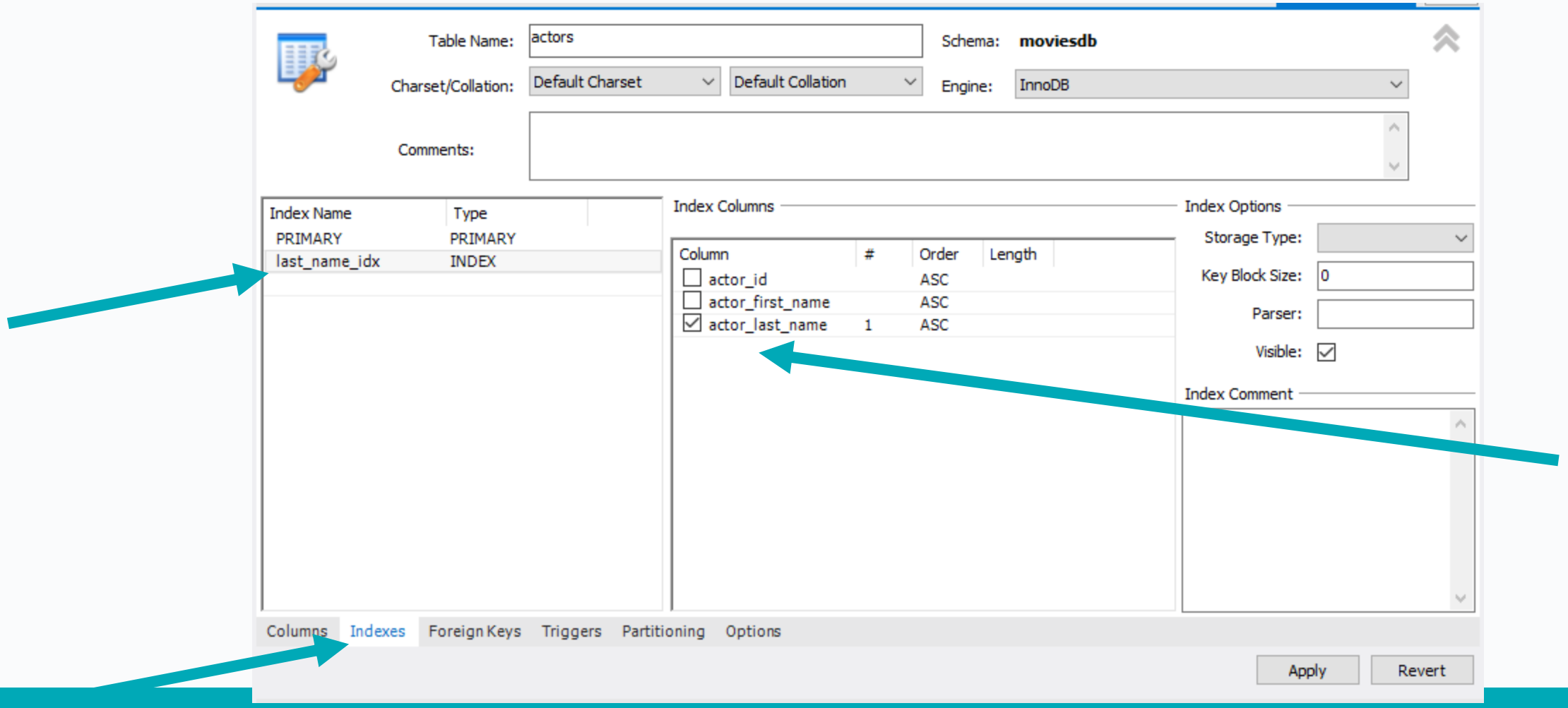
MySQL: Optimisation - Explain

WITHOUT AN INDEX

```
EXPLAIN SELECT * FROM actor WHERE last_name='Berry';
```

	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
▶	1	SIMPLE	actor	NULL	ALL	NULL	NULL	NULL	NULL	200	10.00	Using where

MySQL: Add index to last_name



MySQL: Optimisation

WITH AN INDEX

```
EXPLAIN SELECT * FROM actor WHERE last_name='Berry';
```

	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
▶	1	SIMPLE	actor	NULL	ref	idx_actor_last_name	idx_actor_last_name	77	const	3	100.00	Using where

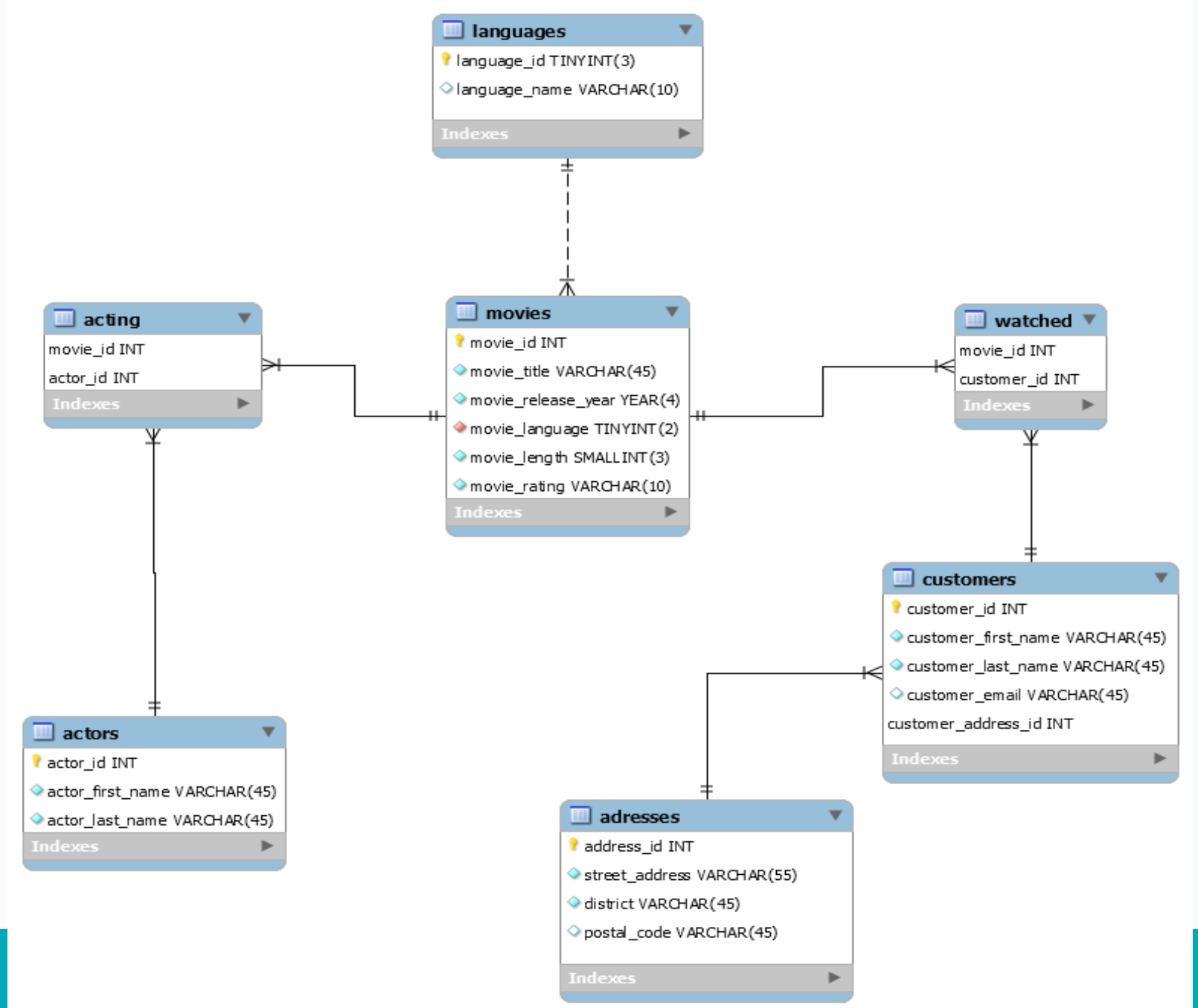
MySQL: Optimisation Summary

- All primary keys are indexes by default (MySQL-InnoDB)
- You should create new indexes when repeated queries use particular columns
- Indexes are best used on columns:
  - That are frequently used in the WHERE part of a query
  - That are frequently used in an ORDER BY part of a query
- Indexes increase efficiency and optimize retrieval time but slow down other tasks such as update or insert

Subqueries

- ***Derived*** table sub-queries
  - Appear in the **`FROM`** clause of a select statement

- ***Expression*** sub-queries
  - Appear in the **`WHERE`** clause of a select statement
    - Sub-queries that return a single value or row
    - Sub-queries that are used to test a Boolean expression


languages
language_id TINYINT(3)
language_name VARCHAR(10)
Indexes
acting
movie_id INT
actor_id INT
Indexes
movies
movie_id INT
movie_title VARCHAR(45)
movie_release_year YEAR(4)
movie_language TINYINT(2)
movie_length SMALLINT(3)
movie_rating VARCHAR(10)
Indexes
watched
movie_id INT
customer_id INT
Indexes
customers
customer_id INT
customer_first_name VARCHAR(45)
customer_last_name VARCHAR(45)
customer_email VARCHAR(45)
customer_address_id INT
Indexes
actors
actor_id INT
actor_first_name VARCHAR(45)
actor_last_name VARCHAR(45)
Indexes
adresses
address_id INT
street_address VARCHAR(55)
district VARCHAR(45)
postal_code VARCHAR(45)
Indexes

Subqueries

- ***Derived*** table sub-queries
  - Appear in the **`FROM`** clause of a select statement

```
SELECT customers_table.email
FROM customers_table INNER JOIN (
                SELECT * FROM address_table
                WHERE
                REGEXP_LIKE(address_table.address,'Boulevard$’)
                )
                AS boul
on customers_table.address_id=boul.address_id;
```

Subqueries

- ***Expression*** sub-queries
  - Appear in the **`WHERE`** clause of a select statement
    - Sub-queries that are used to test a Boolean expression

```
SELECT first_name,last_name FROM customers_table
WHERE customer_id IN (
        SELECT customer_id    FROM watched_table
        INNER JOIN movies using(movie_id)
        INNER JOIN languages ON movies.movie_language= languages.language_id
        WHERE language_name ='Japanese' );
```

Subqueries

- ***Expression*** sub-queries
  - Appear in the **`WHERE`** clause of a select statement
    - Sub-queries that return a **single value or row**

```
SELECT movies.movie_title, movies.movie_length FROM movies
WHERE movies.movie_length < (
                SELECT MIN(movies.movie_length)
                FROM movies INNER JOIN watched_table using(movie_id)
                INNER JOIN customers_table using(customer_id)
                WHERE last_name="Forsythe"                    );
```

Subqueries

- ***Expression*** sub-queries
  - Appear in the **`WHERE`** clause of a select statement
  - C*orrelated* sub-query . Uses data from the outer query in the sub-query.

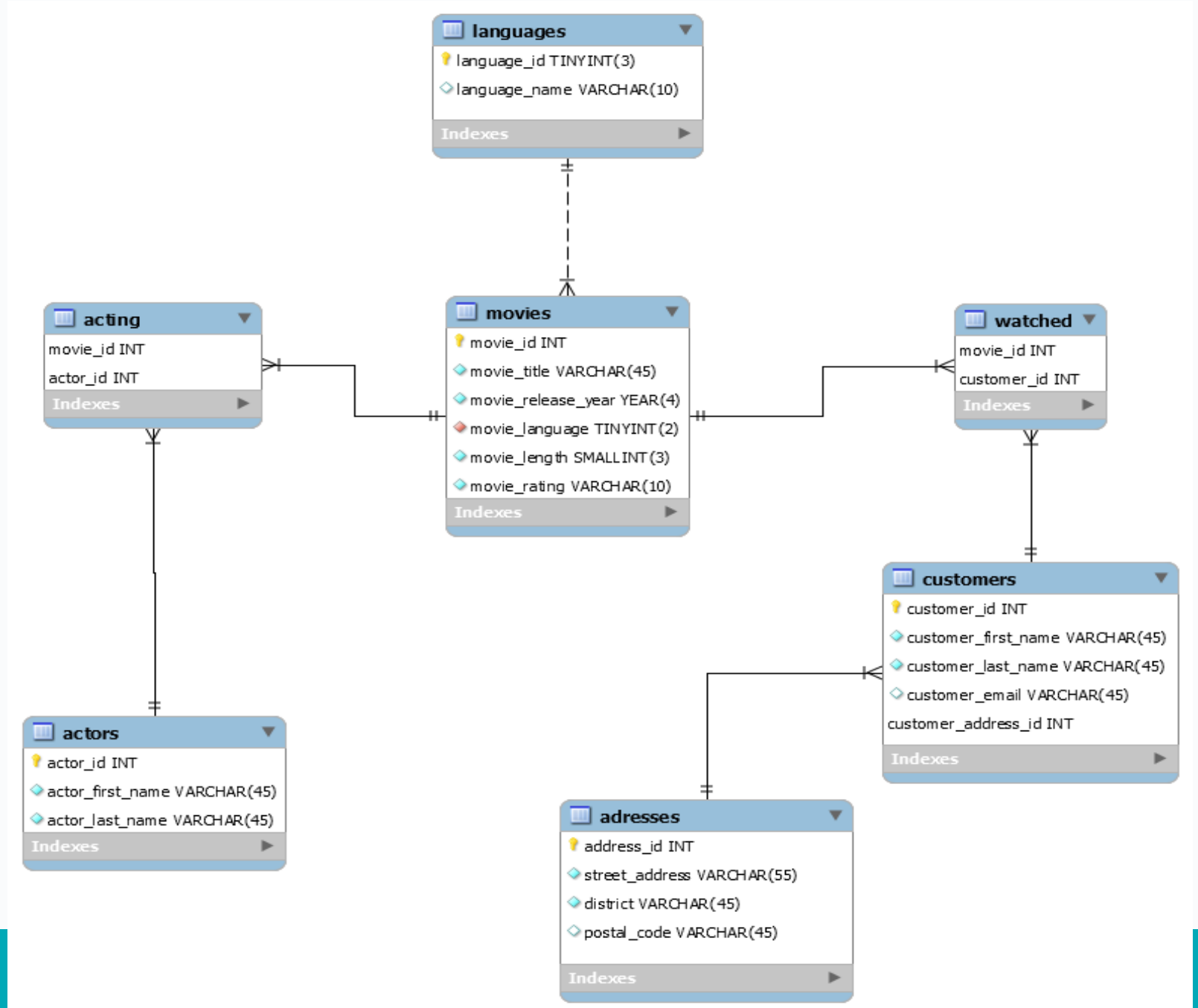
```
SELECT title, rating, length
 FROM movies_table as outer
 WHERE length =
                    (SELECT round(AVG(length),0)
                     FROM movies_table
                      WHERE movies_table.rating=outer.rating
                     );
```

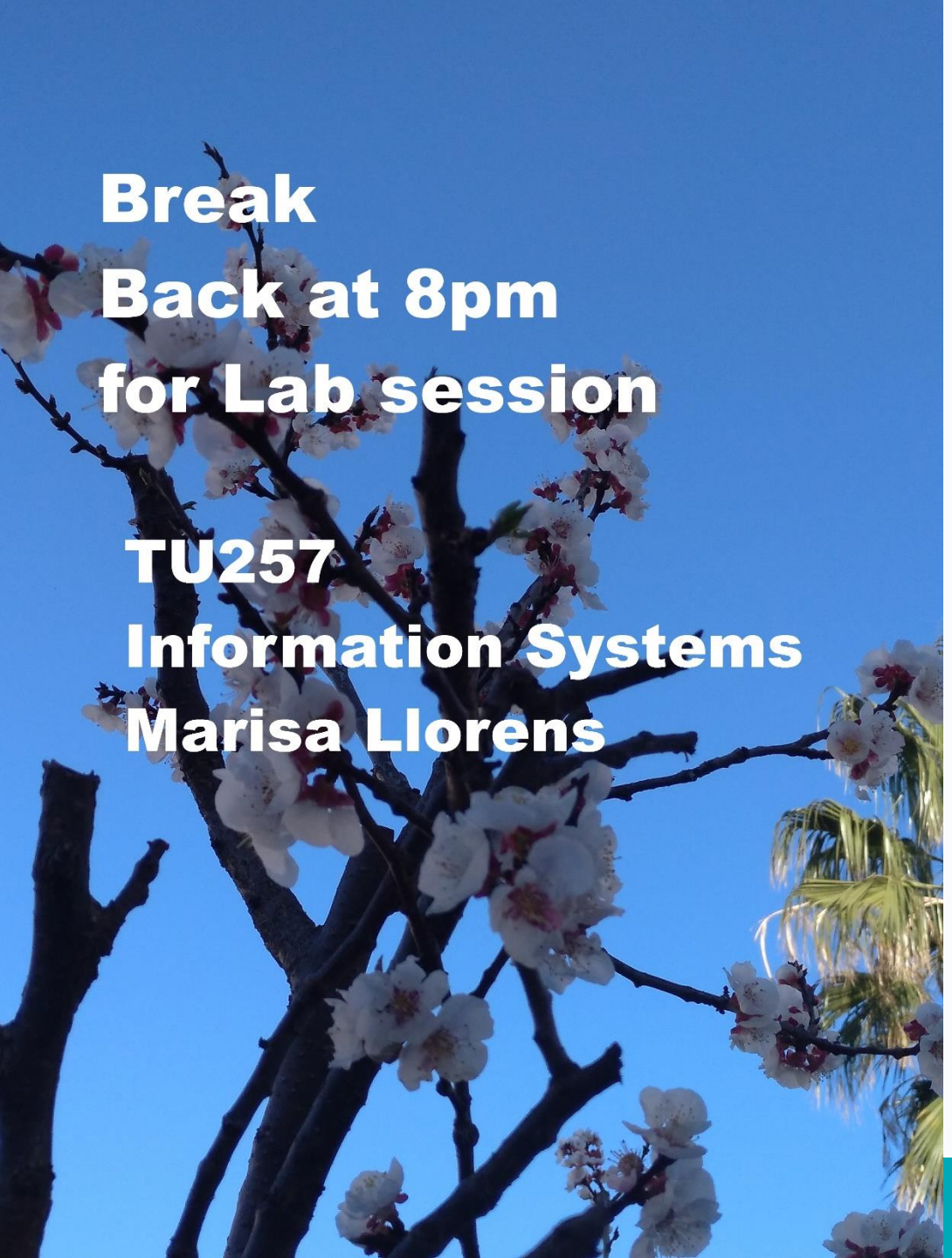
Extra resources

https://www.w3resource.com/sqlite/sqlite-subqueries.php

https://www.w3resource.com/sql/subqueries/understanding-sql-subqueries.php

https://www.sqlitetutorial.net/sqlite-subquery/


languages
language_id TINYINT(3)
language_name VARCHAR(10)
Indexes
acting
movie_id INT
actor_id INT
Indexes
movies
movie_id INT
movie_title VARCHAR(45)
movie_release_year YEAR(4)
movie_language TINYINT(2)
movie_length SMALLINT(3)
movie_rating VARCHAR(10)
Indexes
watched
movie_id INT
customer_id INT
Indexes
customers
customer_id INT
customer_first_name VARCHAR(45)
customer_last_name VARCHAR(45)
customer_email VARCHAR(45)
customer_address_id INT
Indexes
actors
actor_id INT
actor_first_name VARCHAR(45)
actor_last_name VARCHAR(45)
Indexes
adresses
address_id INT
street_address VARCHAR(55)
district VARCHAR(45)
postal_code VARCHAR(45)
Indexes



Break
Back at 8pm
for Lab session

TU257
Information Systems
Marisa Llorens